UT_04.1 Procesamiento y activación de eventos

Un concepto importante en la programación JavaScript son los eventos y, por lo tanto, la programación basada en eventos. En este tema, le mostraremos el principio que hay detrás de los eventos y cómo puede utilizar la programación basada en eventos para reaccionar a los eventos activados por los usuarios dentro de una aplicación (web).

Cuando visita una página web y realiza algunas acciones, se activan varios *eventos* en segundo plano. Por ejemplo, cuando la página se carga por completo; cuando el usuario introduce texto en un campo de texto, selecciona una opción de un cuadro de selección o hace clic en un botón; o cuando los elementos entran o salen del área visible de la pantalla. Dentro de su código JavaScript, tiene la posibilidad de reaccionar a estos eventos. Por ejemplo, puede validar el texto que el usuario ha introducido en un campo de texto o enviar un formulario cuando el usuario hace clic en un botón. El principio de enviar y responder a eventos se denomina *programación basada en eventos.*

1 El concepto de programación basada en eventos

El concepto de programación basada en eventos no es específico de JavaScript, sino que también se utiliza en otros lenguajes de programación. Básicamente, funciona de la siguiente manera (véase también **la figura 1**): por un lado, están los denominados desencadenantes de eventos (también llamados *emisores de eventos*). En el caso de las interfaces gráficas de usuario (UI), por ejemplo, estos pueden ser botones, campos de texto o cualquier otro componente de la UI. Tan pronto como se hace clic en un botón o se introduce texto en un campo de texto, el componente correspondiente (el botón o el campo de texto) activa un *evento* en segundo plano.

Después de que se ha disparado un evento, se coloca en una *cola de eventos*, lo que garantiza que los eventos que se dispararon primero también se gestionen primero. El *bucle de eventos* comprueba continuamente si hay un nuevo evento en la cola de eventos y, si lo hay, el evento correspondiente se reenvía a *los controladores de eventos*.

En JavaScript, estos controladores de eventos son funciones simples que le permiten a usted, el desarrollador, responder al evento activado. Por lo tanto, puede utilizar la programación basada en eventos para expresar cosas como «cuando se hace clic en este botón, se debe llamar a esta o aquella función».

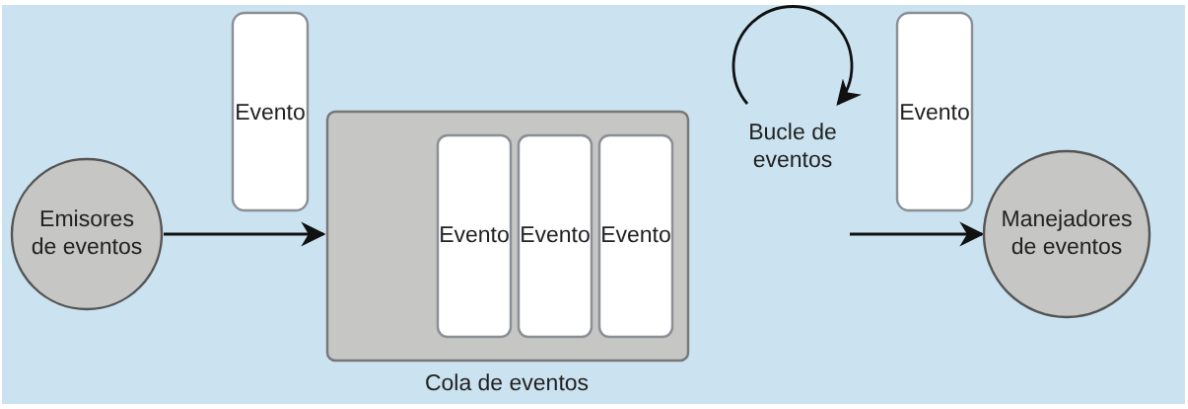


Figura 1 *El principio de los eventos y el manejo de eventos*

Sin embargo, el principio de la programación basada en eventos no se limita a las interacciones con una interfaz de usuario. En el desarrollo de JavaScript, también hay eventos que se activan sin que el usuario interactúe con la interfaz de usuario, es decir, eventos que no son activados por el usuario, sino por programación. En otro tema**,** por ejemplo, presentaremos Node.js, un entorno de ejecución para JavaScript que también utiliza eventos para controlar muchas cosas en segundo plano. Sin embargo, en este tema, primero utilizaremos páginas web (y, por lo tanto, principalmente interfaces de usuario) como ejemplo para mostrarle cómo programar utilizando eventos.

Eventos, pila de llamadas y bucle de eventos

Cuando el usuario interactúa con la página, el navegador coloca una tarea en la cola de eventos; el bucle de eventos la toma y ejecuta el escuchador **solo cuando la pila de llamadas está vacía**. Por eso un escuchador muy lento «congela» la página: mientras se ejecuta, el navegador no puede procesar más eventos. Las promesas (`then`, `await`) usan una cola aparte, la de *microtareas*, que se vacía antes de pasar a la siguiente tarea; se verá con detalle en la unidad dedicada a la asincronía.

2 Responder a eventos

Para activar una función en las interacciones del usuario dentro de una página web, se requieren tres pasos:

1. Seleccionar el *elemento* de la página web que debe responder a la interacción del usuario. Por ejemplo, si desea que se active una función cuando el usuario haga clic en el encabezado de una columna de una tabla, debe seleccionar previamente el elemento `<th>` correspondiente.
2. Especificar el *evento* que se va a capturar. Como verá en un momento, hay muchos tipos diferentes de eventos. Por ejemplo, es posible capturar clics del ratón, pulsaciones de teclas, movimientos del ratón y mucho más. Este paso también se denomina *vincular* un evento a un elemento.
3. Especifique la *función* que se va a llamar. Por último, pero no menos importante, debe definir la función que se va a llamar cuando se produzca el evento correspondiente en el elemento seleccionado. Por ejemplo, ¿se ordenarán los datos de la tabla al hacer clic? ¿Se validará el texto introducido en un campo de texto?

Eventos implícitos

En el desarrollo web también hay eventos que se pueden capturar sin haber seleccionado primero un elemento. Entre ellos se incluyen, por ejemplo, los eventos que se activan cuando una página web se carga por completo, además de muchos otros.

En el tema anterior, aprendiste en detalle cómo seleccionar elementos en una página web, por lo que el paso 1 debería ser pan comido para ti. Veamos, pues, cómo puedes capturar eventos o vincularlos a elementos y cómo definir el código que se ejecutará cuando se active un evento. De hecho, hay tres posibilidades al respecto:

Controladores de eventos HTML
Los controladores de eventos HTML ofrecen una posibilidad de reaccionar ante los eventos, pero

hoy en día ya no se utilizan. Esta opción se basa en atributos HTML especiales para definir qué función se debe llamar para cada evento. La razón por la que ya no se utilizan los controladores de eventos HTML es que esta opción requiere mezclar código HTML y código JavaScript, lo que no se considera una buena opción en términos de mantenibilidad y similares. No obstante, en **la sección 2.1,** le mostraremos los detalles de este tipo de controlador de eventos. No debe utilizar los controladores de eventos HTML usted mismo, pero es posible que los encuentre en aplicaciones web más antiguas, y entonces debe comprender lo que está sucediendo.

Controladores de eventos DOM

La segunda forma de reaccionar a los eventos es a través de los denominados controladores de eventos DOM. Reciben este nombre porque forman parte de la especificación DOM y se definen mediante JavaScript. En comparación con los controladores de eventos HTML, su ventaja es que no se mezclan el código HTML y el código JavaScript. En otras palabras, la lógica de la aplicación (respuesta a eventos) está separada de la estructura de la aplicación (el HTML). Sin embargo, los controladores de eventos DOM también tienen una desventaja o limitación significativa: solo permiten especificar una única función que se llamará para un evento. Te mostraremos los detalles de todo esto en **la sección 2.2.**

Escuchadores de eventos DOM

Por último, pero no menos importante, existe una tercera opción para reaccionar a los eventos: los denominados escuchadores de eventos DOM. Estos se introdujeron en 2000 como parte de la segunda versión de la especificación DOM (**www.w3.org/TR/DOM-Level-2-Events/Overview.html**) y, desde entonces, han sido el estándar para el manejo de eventos y la forma recomendada de responder a ellos. La razón: mientras que en el caso de los controladores de eventos solo es posible registrar un único controlador de eventos para un evento, los escuchadores de eventos permiten registrar varios escuchadores de eventos para el mismo evento. Te mostraremos cómo definir escuchadores de eventos en **la sección 2.3.**

Por qué se evitan los controladores HTML

Además de mezclar HTML y JavaScript, los controladores de eventos HTML tienen dos inconvenientes más: una *Content Security Policy* estricta (que muchos sitios activan por seguridad) los bloquea, y si el script se carga como módulo ES6 (`<script type="module">`) las funciones dejan de ser globales, por lo que `onclick="miFuncion()"` ya no las encuentra. Los frameworks actuales (`onClick` en React, `@click` en Vue, `(click)` en Angular) no son controladores HTML: internamente registran escuchadores de eventos.

Definición de términos

Los términos *«controlador de eventos»* y *«escuchador de eventos»* se utilizan a menudo como sinónimos en la literatura. Sin embargo, en el caso de JavaScript (como se ha descrito anteriormente), se hace una distinción entre ambos términos. Es decir, una función que se ha definido como controlador de eventos HTML o controlador de eventos DOM se denominará en adelante controlador de eventos, y una función que se ha definido como escuchador de eventos se denominará, en consecuencia, escuchador de eventos.

La secuencia esencial cuando se produce un evento se muestra en **la figura 2**. En primer lugar, se comprueba si hay un controlador de eventos o un escuchador de eventos definido para el evento.

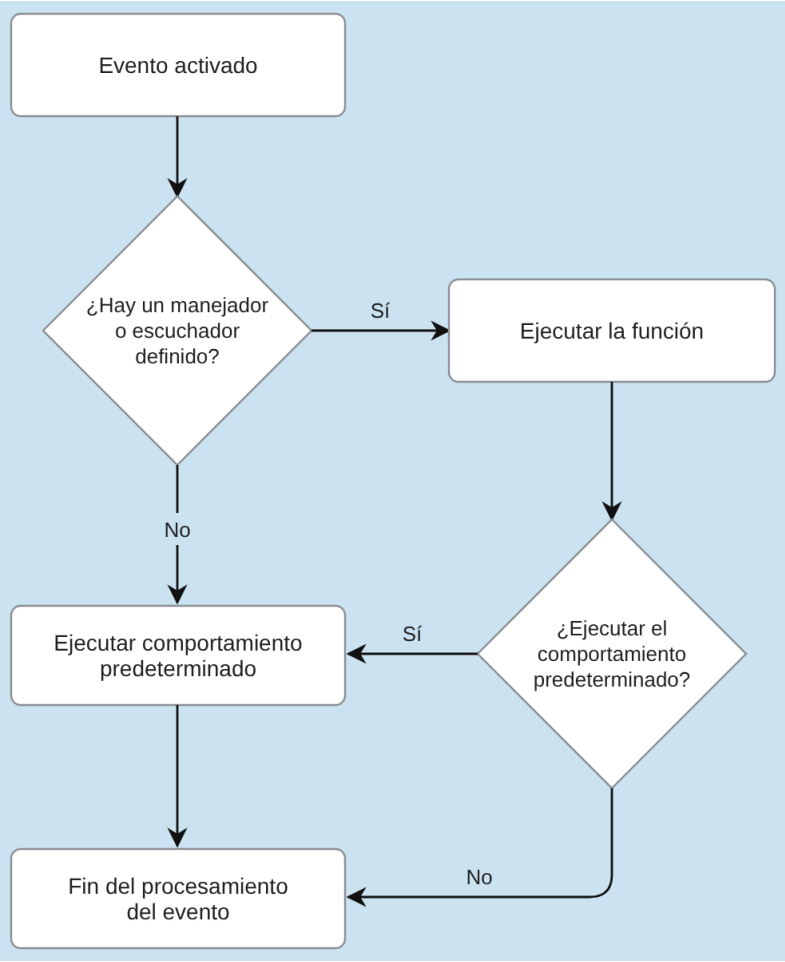


***Figura 2** Procedimiento de gestión de eventos*

Si es así, se ejecuta el código correspondiente. Si, por el contrario, no se han definido controladores de eventos/escuchadores de eventos, se activan las acciones predeterminadas del navegador asociadas al evento correspondiente. Dentro de un controlador de eventos/escuchador de eventos, también tiene la posibilidad de desactivar estas acciones predeterminadas del navegador para un evento. Le mostraremos cómo hacerlo en **la sección 4**.

2.1 Definición de un controlador de eventos mediante HTML

Tomemos un caso de uso sencillo como ejemplo de gestión de eventos: la validación de un campo de formulario que solo debe aceptar números positivos. Para empezar, lo único que hay que comprobar es que no se haya especificado ningún valor negativo. La validación debe activarse cada vez que el usuario salga del campo de formulario correspondiente (es decir, cuando el usuario haga clic fuera del campo de formulario).

Validación nativa en HTML5

Hasta hace unos años, la validación de los campos de formulario solo era posible mediante JavaScript. Hoy HTML incluye atributos como `required`, `min`, `max`, `pattern` o `type="number"`, y el navegador ofrece la *Constraint Validation API* (`checkValidity()`, `reportValidity()`, `setCustomValidity()`, la propiedad `validity` y el evento `invalid`), además de pseudoclases CSS

como `:user-invalid`. Para el caso de este tema bastaría con `<input type="number" min="0">`; aun así, usamos JavaScript porque nos permite aprender la gestión de eventos y validar con reglas propias.

Validación del lado del cliente y del lado del servidor

Una nota más: en una aplicación de producción en la que los usuarios pueden rellenar y enviar formularios, debe validar los datos introducidos no solo en el lado del cliente, sino también en el lado del servidor. Básicamente, la validación del lado del cliente se realiza para facilitar el uso y alertar al usuario de cualquier error. La validación del lado del servidor, sin embargo, garantiza que los datos transmitidos no hayan sido manipulados (maliciosamente) por el usuario.

Ahora le mostraremos cómo se puede implementar este caso de uso utilizando un controlador de eventos HTML. Como se ha mencionado anteriormente, los controladores de eventos HTML se definen dentro del código HTML utilizando atributos HTML. Los atributos HTML siempre llevan el nombre del evento precedido por `on`. En el ejemplo, el atributo HTML es `onblur` porque el evento `blur` se activa cuando se sale de un elemento del formulario.

El valor del atributo es la función que se debe llamar cuando se activa el evento. En resumen, esto puede dar como resultado el siguiente atributo HTML si la función se llama `comprobarEdadNoNegativa`, como en el ejemplo:

```
onblur="comprobarEdadNoNegativa()"
```

Puede ver el código HTML completo en **el Listado 1**. El campo de entrada con el ID `de edad` se utiliza para introducir la edad y el elemento `<div>` con el ID `de salida` sirve como contenedor para los mensajes de validación.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Controlador de eventos HTML</title>
  <link rel="stylesheet" href="estilos/principal.css">
  <script src="scripts/principal.js" defer></script>
</head>
<body>
  <div>
    <label for="edad">Edad:</label>
    <input id="edad" type="number" value="0" onblur="comprobarEdadNoNegativa()">
  </div>
  <div id="salida"></div>
</body>
</html>
```

***Listado 1** Definición de un controlador de eventos HTML que se invoca al salir de un campo de entrada*

La función llamada (es decir, el controlador de eventos) se muestra en **el Listado 2**. En realidad, aquí no ocurre nada que no se haya visto ya en el tema anterior: en primer lugar, el campo de entrada y el contenedor se determinan por su ID y se asignan a una variable. A continuación, se lee y se comprueba el valor del campo de entrada. Si el valor es inferior a 0 (y, por lo tanto, no es válido), se escribe el mensaje de error correspondiente en el contenedor. Por otro lado, si el valor

es 0 o superior (y, por lo tanto, válido), se borra el contenido de texto del contenedor (es decir, se le asigna una cadena vacía). Esto último es necesario para borrar cualquier mensaje de error si se introduce un valor válido en el campo de entrada después de un valor no válido.

```
function comprobarEdadNoNegativa() {
  const salida = document.getElementById('salida'); // Contenedor para el mensaje
  const campo = document.getElementById('edad');  // Campo de entrada de la edad
  const edad = Number(campo.value);  // Valor actual de la edad
  if (edad < 0) {  // Si el valor es negativo...
    salida.textContent = 'La edad no puede ser negativa.'; // ...muestra un error
  } else {  // ...en caso contrario...
    salida.textContent = '';  // ...elimina el mensaje.
  }
}
```

***Listado 2** La función que se llamará como controlador de eventos*

Si no desea activar la validación tan pronto como se salga del campo de entrada, sino solo al pulsar un botón, por ejemplo, el código se puede adaptar con relativa facilidad, como se muestra en **el Listado 3**. Aquí, se ha eliminado el controlador de eventos para el evento `blur` del campo de texto y se ha insertado un botón adicional para el que se ha definido el controlador de eventos para el evento `click` (que se activa cuando se pulsa el botón correspondiente).

```
<input id="edad" type="number" value="0">
<button type="button" onclick="comprobarEdadNoNegativa()">Comprobar edad</button>
```

***Listado 3** Definición de un controlador de eventos HTML que se activa al hacer clic en un botón*

Aunque los controladores de eventos HTML parecen relativamente sencillos de usar a primera vista, ahora se consideran obsoletos porque mezclan código HTML y código JavaScript, lo que puede resultar confuso en aplicaciones web de gran tamaño. En su lugar, es mejor utilizar controladores de eventos DOM, que le mostraremos en la siguiente sección, o (mejor aún) escuchadores de eventos, que presentaremos en **la sección 2.3**.

2.2 Definición de un controlador de eventos mediante JavaScript

Los controladores de eventos DOM resuelven el problema de la mezcla de código HTML y JavaScript. Como se muestra en **el Listado 4**, el código HTML permanece libre de llamadas JavaScript. El elemento que debe dar lugar a la llamada de una función para un evento determinado se define en el código JavaScript.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Controlador de eventos DOM</title>
  <link rel="stylesheet" href="estilos/principal.css">
  <script src="scripts/principal.js" defer></script>
</head>
<body>
  <div>
    <label for="edad">Edad:</label>
    <input id="edad" type="number" value="0">
```

```
  </div>
  <div id="salida"></div>
</body>
</html>
```

Listado 4 *Cuando se utilizan controladores de eventos DOM, el HTML permanece libre de código JavaScript*

El funcionamiento se muestra en **el Listado 5**. En primer lugar, el campo de entrada para el que se va a definir el controlador de eventos se determina de nuevo en función del ID. Para la propiedad `onblur` de este objeto, se define el nombre de la función que se va a llamar. Tenga en cuenta que, a diferencia de los controladores de eventos HTML, no hay ninguna llamada a la función, sino solo el nombre de la función (es decir, sin el par de paréntesis). Si escribieras `campo.onblur = comprobarEdadNoNegativa()` en lugar de `campo.onblur = comprobarEdadNoNegativa,` el valor de retorno de la llamada a la función `comprobarEdadNoNegativa()` se definiría como un controlador de eventos. En este caso, esto no es lo que queremos.

```
function iniciar() {
  const campo = document.getElementById('edad'); // (1) Obtener elemento
  campo.onblur =  // (2) Definir evento
    comprobarEdadNoNegativa;  // (3) Definir controlador de eventos
}
window.onload = iniciar;
```

Listado 5 *Definición de un controlador de eventos DOM*

Controladores de eventos para cargar una página web

En **el Listado 5**, también se puede ver que la propiedad `onload` del objeto `window` define un controlador de eventos que se invoca cuando se carga la página web. Sin este controlador de eventos, la función `iniciar()` no se invocaría y, por lo tanto, no se registraría ningún controlador de eventos en el elemento del formulario.

Los controladores de eventos DOM tienen una pequeña desventaja, como se mencionó anteriormente: solo se puede definir un controlador de eventos por evento en un elemento. Por ejemplo, si quisieras ampliar la validación anterior para llamar no solo a una función que comprueba si el valor introducido es no negativo, sino también a otra función que comprueba si el valor introducido es un número (llamémosla `comprobarEdadEsNumero()`), esto no sería posible, como se muestra en **el Listado 6**. Este listado define primero la función `comprobarEdadNoNegativa` como un controlador de eventos para la propiedad `onblur` y, a continuación, la función `comprobarEdadEsNumero`. Sin embargo, esta última solo garantiza que se restablezca la propiedad `onblur` y se sobrescriba el valor `comprobarEdadNoNegativa`. El resultado es que solo se llama a la función `comprobarEdadEsNumero`, pero no a la función `comprobarEdadNoNegativa`.

```
function iniciar() {
  const campo = document.getElementById('edad');
  campo.onblur = comprobarEdadNoNegativa;
  campo.onblur = comprobarEdadEsNumero; // Aquí se sobrescribe el controlador anterior.
```

```
}
window.onload = iniciar;
```

***Listado 6** Solo se puede definir un controlador de eventos por evento*

Esto se puede solucionar, al menos en lo que respecta al ejemplo, utilizando funciones anónimas. Como se muestra en **el Listado 7**, las funciones `comprobarEdadNoNegativa` y `comprobarEdadEsNumero` se pueden encapsular simplemente dentro de una función anónima, que luego se define como un controlador de eventos.

```
function iniciar() {
  const campo = document.getElementById('edad');
  campo.onblur = function () { // función anónima...
    comprobarEdadNoNegativa(); // ...en la que se llama a esta función...
    comprobarEdadEsNumero();  // ...y también a esta otra
  };
}
window.onload = iniciar;
```

***Listado** 7 Las funciones anónimas pueden eludir parcialmente el problema descrito, pero son relativamente inflexibles*

Aunque este enfoque funciona para el ejemplo que nos ocupa, si es posible, se deben definir las funciones como escuchadores de eventos en lugar de como controladores de eventos. En la siguiente sección le mostraremos cómo hacerlo.

2.3 Definición de escuchadores de eventos

La desventaja de los controladores de eventos mencionada en la sección anterior no se aplica a los escuchadores de eventos ni al método `addEventListener()` relacionado, que está disponible en cualquier objeto que pueda recibir eventos (elementos, `document`, `window`...). Como parámetros, espera el nombre del evento al que se va a responder y el escuchador de eventos, es decir, la función que se va a llamar cuando se produzca el evento. Opcionalmente, se puede especificar un tercer parámetro, un objeto de opciones (en código antiguo, un booleano), para influir en *el flujo del evento* (los detalles se explican en la **sección 4**) y en la vida del escuchador (véase el recuadro *Opciones de addEventListener()*, más abajo).

El listado 8 muestra un ejemplo que hace lo mismo que el código de las secciones anteriores: llama a la función `comprobarEdadNoNegativa()` al salir del campo del formulario correspondiente para introducir la edad. El primer parámetro es el valor `de blur` (sin `on`; no utilice `onblur`); el segundo parámetro es el nombre de la función, `comprobarEdadNoNegativa()`.

```
function iniciar() {
  const campo = document.getElementById('edad'); // Obtener elemento
  campo.addEventListener(  // Registrar el escuchador de eventos
    'blur',  // Nombre del evento
    comprobarEdadNoNegativa  // Función escuchadora (sin paréntesis)
  );
}
document.addEventListener('DOMContentLoaded', iniciar);
```

***Listado 8** Definición de un escuchador de eventos*

Escuchadores de eventos para cargar el contenido del documento

Aunque ya lo hemos mencionado anteriormente, volvamos a referirnos brevemente al evento `DOMContentLoaded` en este punto. Se activa cada vez que se carga todo el árbol DOM de una página web y es perfecto para ejecutar código inicial en una página web, como registrar escuchadores de eventos (o controladores de eventos). En los ejemplos de código utilizamos este evento en la mayoría de los casos para ejecutar código dentro de una página web. En **el Listado 9** se muestra un ejemplo sencillo**.**

```
function iniciar() {
  console.log('Documento cargado');
}

document.addEventListener('DOMContentLoaded', iniciar);
```

Listado 9 *Definición de un escuchador de eventos que se ejecuta cuando se carga la página web*

Si desea definir un escuchador de eventos para el evento de carga (análogo a `window.onload`), proceda como se describe en **el Listado 10**. Fíjese en que el escuchador se registra en `window`: el evento `load` se lanza sobre la ventana, por lo que `document.addEventListener('load', …)` no funciona. La diferencia entre el evento `DOMContentLoaded` y el evento `load` es que el primero se activa cuando se ha cargado y analizado todo el documento HTML (es decir, el árbol DOM), mientras que el segundo se activa cuando, además, se han cargado todos los recursos externos, como hojas de estilo, imágenes, etc. Por cierto: no existe ninguna propiedad comparable en el objeto `window` que permita definir un controlador de eventos para `DOMContentLoaded`. Este evento solo puede captarse mediante escuchadores de eventos.

```
function iniciar() {
  console.log('Documento y todos los recursos cargados');
}
window.addEventListener('load', iniciar); // load se escucha en window, no en document
```

Listado 10 *Definición de un escuchador de eventos que se ejecuta cuando se ha cargado la página web, incluidos todos los recursos*

Cargar el script como módulo ES6

Con scripts de tipo módulo (`<script type="module" src="…">`) el código se ejecuta una vez analizado el documento (equivale a `defer`), por lo que ya no hace falta esperar a `DOMContentLoaded` para acceder a los elementos. Además, cada módulo tiene su propio ámbito y puede importar y exportar funciones, lo que permite probar y reutilizar las validaciones por separado.

```
<script type="module" src="scripts/principal.js"></script>

// scripts/validaciones.js
export const esEdadNoNegativa = (edad) => edad >= 0;
export const esNumero = (valor) => valor !== '' && Number.isFinite(Number(valor));

// scripts/principal.js
import { esEdadNoNegativa, esNumero } from './validaciones.js';
```

```
const campoEdad = document.getElementById('edad');
const salida = document.getElementById('salida');

campoEdad.addEventListener('blur', () => {
  const valor = campoEdad.value;
  if (!esNumero(valor)) {
    salida.textContent = 'La edad debe ser un número.';
  } else if (!esEdadNoNegativa(Number(valor))) {
    salida.textContent = 'La edad no puede ser negativa.';
  } else {
    salida.textContent = '';
  }
});
```

Listado 11 *Validación de la edad con módulos ES6 (dos archivos)*

Código antiguo: el método attachEvent()

Si alguna vez se encuentra el método `attachEvent()` en el código de otra persona, no se sorprenda: era un método propio de Internet Explorer (navegador jubilado en 2022) que no forma parte de ningún estándar. Hoy no debe utilizarse.

Opciones de addEventListener()

El tercer parámetro de `addEventListener()` admite hoy un objeto de opciones (el booleano clásico equivale a `{ capture: true }` o `{ capture: false }`). Las más útiles son:

- `capture`: registra el escuchador para la fase de captura en lugar de la de burbujeo (**sección 4**).
- `once`: el escuchador se ejecuta como máximo una vez y se elimina solo.
- `passive`: promete que el escuchador no llamará a `preventDefault()`, lo que permite al navegador optimizar el desplazamiento; se recomienda en `scroll`, `wheel` y `touch`*.
- `signal`: recibe un `AbortSignal`; al llamar a `abort()` sobre su `AbortController` se eliminan todos los escuchadores registrados con esa señal.

```
const boton = document.getElementById('guardar');
const controlador = new AbortController();

// Se ejecuta una sola vez y se elimina automáticamente
boton.addEventListener('click', () => {
  console.log('Primer clic');
}, { once: true });

// Se podrá eliminar sin guardar la referencia a la función
boton.addEventListener('click', () => {
  console.log('Guardando...');
}, { signal: controlador.signal });

// Al pulsar "cerrar" se eliminan todos los escuchadores asociados a la señal
document.getElementById('cerrar').addEventListener('click', () => {
  controlador.abort();
});
```

***Listado 12** Las opciones once y signal de addEventListener()*

2.4 Definición de múltiples escuchadores de eventos

Con el método `addEventListener()`, ahora también es muy fácil registrar varias funciones para un mismo evento (lo cual solo es posible con los controladores de eventos mediante el desvío de utilizar una nueva función, normalmente anónima).

El listado 13 muestra un ejemplo en el que se registran hasta tres funciones diferentes como escuchadores de eventos para el evento `blur` en el campo correspondiente del formulario: la función `borrarMensaje()` garantiza que se borre el mensaje, la función `comprobarEdadNoNegativa()` comprueba que la edad no sea negativa y `la función comprobarEdadEsNumero()` garantiza que el valor introducido sea un número.

```
function comprobarEdadNoNegativa() {
  const campo = document.getElementById('edad');  // Campo de entrada de la edad
  const edad = Number(campo.value);  // Valor actual de la edad
  if (edad < 0) {  // Si el valor es negativo...
    mostrarMensaje('La edad no puede ser negativa.'); // ...muestra una advertencia.
  }
}

function comprobarEdadEsNumero() {
  const campo = document.getElementById('edad');
  const valor = campo.value;  // Valor actual (texto)
  if (valor === '' || !Number.isFinite(Number(valor))) { // Si no es un número...
    mostrarMensaje('La edad debe ser un número.');  // ...muestra un mensaje.
  }
}

function borrarMensaje() {
  mostrarMensaje('');
}

function mostrarMensaje(mensaje) {
  const salida = document.getElementById('salida');
  salida.textContent = mensaje;
}

function iniciar() {
  const campo = document.getElementById('edad');  // Obtener elemento
  campo.addEventListener('blur', borrarMensaje);
  campo.addEventListener('blur', comprobarEdadNoNegativa);
  campo.addEventListener('blur', comprobarEdadEsNumero);
}

document.addEventListener('DOMContentLoaded', iniciar);
```

***Listado 13** Definición de múltiples escuchadores de eventos*

Nota

El uso de escuchadores de eventos o del método `addEventListener()` es mucho más flexible en comparación con los controladores de eventos, ya que permite especificar no solo una única función, sino cualquier número de funciones.

2.5 Pasar argumentos a los escuchadores de eventos

A veces, es posible que desee pasar argumentos a la función que ha registrado como escuchador de eventos. Sin embargo, esto no es posible de forma inmediata, ya que solo se pasa la función en sí al método `addEventListener()`. Una vez más, la solución es utilizar funciones anónimas, como se puede ver en **el Listado 14**.

Las funciones `comprobarEdadNoNegativa()` y `comprobarEdadEsNumero()` ya no acceden al campo del formulario con la edad, sino que esperan un valor numérico como parámetro para comprobar. Esto hace que ambas funciones sean mucho más reutilizables, ya que ya no acceden directamente al árbol DOM y, en principio, también podrían utilizarse en un contexto diferente.

Además, el código para acceder al árbol DOM o al campo del formulario se ha trasladado a la nueva función `obtenerEdad()`. Dentro de los escuchadores de eventos (funciones anónimas en ambos casos), el valor devuelto por `obtenerEdad()` se pasa a continuación a los dos métodos de comprobación.

```
function comprobarEdadNoNegativa(edad) {
  if (edad < 0) {
    mostrarMensaje('La edad no puede ser negativa.');
  }
}

function comprobarEdadEsNumero(edad) {
  if (!Number.isFinite(edad)) {
    mostrarMensaje('La edad debe ser un número.');
  }
}

function borrarMensaje() {
  mostrarMensaje('');
}

function mostrarMensaje(mensaje) {
  const salida = document.getElementById('salida');
  salida.textContent = mensaje;
}

function obtenerEdad() {
  const campo = document.getElementById('edad');
  return campo.valueAsNumber; // NaN si el campo está vacío o no es válido
}

function iniciar() {
  const campo = document.getElementById('edad');
  campo.addEventListener('blur', borrarMensaje);
```

```
  campo.addEventListener(
    'blur',
    function () {  // función anónima
      const edad = obtenerEdad();  // obtener el valor de la edad
      comprobarEdadNoNegativa(edad);  // llamar a la función real
    }
  );
  campo.addEventListener(
    'blur',
    function () {  // función anónima
      const edad = obtenerEdad();  // obtener el valor de la edad
      comprobarEdadEsNumero(edad);  // llamar a la función real
    }
  );
}
document.addEventListener('DOMContentLoaded', iniciar);
```

Listado 14 *Pasar parámetros a los escuchadores de eventos*

Escuchadores de eventos como funciones flecha

Tampoco hay ninguna razón por la que no se deban definir los escuchadores de eventos como funciones flecha. Por lo tanto, el ejemplo anterior se puede reformular como se muestra en **el Listado 15**.

```
campo.addEventListener(
  'blur',
  () => {  // función flecha anónima
    const edad = obtenerEdad();  // obtener el valor de la edad
    comprobarEdadNoNegativa(edad);  // llamar a la función real
  }
);
campo.addEventListener(
  'blur',
  () => {  // función flecha anónima
    const edad = obtenerEdad();  // obtener el valor de la edad
    comprobarEdadEsNumero(edad);  // llamar a la función real
  }
);
```

Listado 15 *Los escuchadores de eventos también se pueden definir como funciones flecha*

this, target y currentTarget

En una función tradicional `this` coincide con el elemento en el que se registró el escuchador; en una función flecha no (hereda el `this` del contexto). Por eso conviene acostumbrarse a `evento.currentTarget` (elemento en el que está registrado el escuchador) y `evento.target` (elemento en el que se originó el evento), que funcionan en ambos casos.

2.6 Eliminación de escuchadores de eventos

Los escuchadores de eventos que se han añadido a un elemento para un evento también se pueden eliminar de nuevo utilizando el método `removeEventListener()`. Como parámetros, se pasa el nombre del evento y el escuchador de eventos que se va a eliminar.

En **el Listado 16** se muestra un ejemplo. El ejemplo de validación de edad anterior se ha ampliado aquí con una casilla de verificación que se puede utilizar para desactivar o reactivar la validación. Para ello, se asigna a la casilla de verificación un escuchador de eventos para el evento de `'change'` (que se activa cada vez que se habilita o deshabilita la casilla de verificación) y, dentro de este escuchador de eventos, se utiliza la propiedad `checked` para comprobar si la casilla de verificación está habilitada o deshabilitada. En el primer caso, se añaden los escuchadores de eventos `comprobarEdadNoNegativa()` y `comprobarEdadEsNumero()`, y en el segundo caso se eliminan mediante el método `removeEventListener()`.

```
/* Aquí van las demás funciones (las del ejemplo de múltiples escuchadores). */
function iniciar() {
  const campo = document.getElementById('edad');
  campo.addEventListener('blur', borrarMensaje);
  campo.addEventListener('blur', comprobarEdadNoNegativa);
  campo.addEventListener('blur', comprobarEdadEsNumero);

  const casilla = document.getElementById('validacion');
  casilla.addEventListener('change', () => {
    if (casilla.checked) {
      campo.addEventListener('blur', comprobarEdadNoNegativa);
      campo.addEventListener('blur', comprobarEdadEsNumero);
    } else {
      borrarMensaje();
      campo.removeEventListener('blur', comprobarEdadNoNegativa);
      campo.removeEventListener('blur', comprobarEdadEsNumero);
    }
  });
}
document.addEventListener('DOMContentLoaded', iniciar);
```

***Listado 16** Eliminación de escuchadores de eventos*

removeEventListener() y las funciones anónimas

Para eliminar un escuchador hay que pasar exactamente la misma función con la que se registró (y la misma fase). Por eso una función anónima o flecha registrada directamente no puede eliminarse con `removeEventListener()`; en esos casos se utiliza la opción `signal`.

2.7 Acceso a la información de un evento

Dentro de una función que se ha registrado como controlador de eventos o como escuchador de eventos, se puede acceder a cierta información del evento desencadenado a través de un parámetro que se pasa al llamar a la función correspondiente (pero que hemos omitido en los listados anteriores por razones didácticas). Cada evento está representado por un tipo de objeto específico; por ejemplo, los eventos del ratón por el tipo de objeto `MouseEvent` y los eventos del teclado por el tipo de objeto `KeyboardEvent`. Todos estos tipos de objetos tienen un tipo (superior) común del que derivan: el tipo `Event`. Las propiedades que proporciona este tipo se enumeran en **la Tabla 1.** (Los tipos de objetos individuales, como `MouseEvent` y `KeyboardEvent`, siguen proporcionando otras propiedades, algunas de las cuales se comentarán más adelante en este tema).

Propiedad	Descripción
`bubbles`	Contiene una indicación de si un evento se mueve hacia arriba en el árbol DOM (**Sección 4**).
`cancelable`	Contiene una indicación de si un evento se puede cancelar o no (**Sección 4.3**).
`currentTarget`	Contiene una referencia al objetivo actual del evento (**Sección 4**).
`defaultPrevented`	Contiene una indicación de si se ha llamado al método `preventDefault()` en el evento (**Sección 4.3**).
`eventPhase`	Contiene un valor numérico que representa la fase en la que se encuentra actualmente el evento (**Sección 4.1**). Valores posibles: 0 (o `Event.NONE`) 1 (o `Event.CAPTURING_PHASE`) 2 (o `Event.AT_TARGET`) 3 (o `Event.BUBBLING_PHASE`)
`target`	Contiene una referencia al objetivo original del evento (**Sección 4**).
`timeStamp`	Contiene el momento en que se activó el evento: milisegundos (con decimales) desde que se cargó la página.
`type`	Contiene el nombre del evento.
`isTrusted`	Indica si el evento fue activado por el navegador (por ejemplo, al hacer clic en un botón) o por código JavaScript (**Sección 5**).
`composed`	Indica si el evento puede atravesar los límites de un Shadow DOM (componentes web).
`composedPath()`	Método que devuelve el arreglo de elementos por los que pasa el evento.

***Tabla 1** Propiedades del evento*

En **el Listado 17** se muestra un ejemplo de código para acceder a dicho objeto de evento. Aquí se puede ver una función que se registra como escuchador de eventos para el evento `de click` de un botón. Cuando se invoca, se le pasa como parámetro un objeto de tipo `MouseEvent` (que, por lo tanto, también es implícitamente de tipo `Event`). Dentro de la función, se muestran las diversas propiedades de este objeto (aunque en este punto hemos omitido las propiedades definidas por el tipo `MouseEvent`).

```
function botonPulsado(evento) {
  console.log(evento.bubbles);  // true
  console.log(evento.cancelable);  // true
```

```
  console.log(evento.currentTarget);  // <button>
  console.log(evento.defaultPrevented); // false
  console.log(evento.eventPhase);  // 2
  console.log(evento.target);  // <button>
  console.log(evento.timeStamp);  // p. ej., 5873.4 (ms desde la carga de la página)
  console.log(evento.type);  // «click»
  console.log(evento.isTrusted);  // true
}

function iniciar() {
  const boton = document.getElementById('boton'); // Obtener botón
  boton.addEventListener('click', botonPulsado);  // Registrar el escuchador de eventos
}

document.addEventListener('DOMContentLoaded', iniciar);
```

***Listado 17** Acceso al objeto de evento*

Como se ha mencionado anteriormente, el tipo del objeto de evento difiere en función del tipo de evento desencadenado. Los tipos de eventos disponibles —y, por lo tanto, también los tipos de objetos disponibles— se detallarán en las siguientes secciones.

2.8 Eventos y módulos ES6

Los ejemplos de esta unidad pueden organizarse con módulos ES6: cada archivo exporta sus funciones y un único módulo de entrada las importa y registra los escuchadores (véase el **Listado 11**). Al ser los módulos diferidos y de ámbito propio, el código queda más ordenado y no es necesario declarar funciones globales.

La **tabla 2** resume las diferencias entre las tres formas de registrar la respuesta a un evento.

Característica	Atributo HTML	Propiedad DOM (`onclick`)	`addEventListener()`
Separa HTML y JavaScript	No	Sí	Sí
Varios por evento	No	No (se sobrescribe)	Sí
Fase de captura	No	No	Sí (`capture`)
Eliminación	No	Asignar `null`	`removeEventListener()` o `signal`
Compatible con CSP estricta	No	Sí	Sí
Funciona con módulos ES6	No (la función no es global)	Sí	Sí
Opciones `once` y `passive`	No	No	Sí

***Tabla 2** Comparativa de las tres formas de registrar eventos*

3 Los diferentes tipos de eventos

Dentro de una aplicación web, puede haber diferentes tipos de eventos. Básicamente, se pueden distinguir los siguientes tipos:

- Eventos al interactuar con el ratón (**sección 3.1**)
- Eventos al interactuar con el teclado o con campos de texto (**sección 3.2**)
- Eventos al trabajar con formularios (**sección 3.3**)
- Eventos al enfocar elementos (**Sección 3.4**)
- Eventos relacionados con la interfaz de usuario (**Sección 3.5**)
- Eventos en dispositivos móviles (**Sección 3.6**)

Otros eventos

Además de los eventos que presentaremos en este tema, hay otros muchos con los que te encontrarás al desarrollar con JavaScript. Algunos de ellos los trataremos más adelante.

Se puede encontrar una buena descripción general de los distintos eventos en Referencia de Eventos - API web | MDN

3.1 Eventos al interactuar con el ratón

La forma clásica de interactuar con las páginas web es mediante el ratón. Cada vez que un usuario mueve el ratón o pulsa los botones del ratón, se activa un evento en segundo plano. Los eventos relacionados se muestran en **la tabla 3**. Además de los simples clics del ratón (evento `click`), hay un evento para el doble clic (`dblclick`) y para cuando se ha abierto el menú contextual (`contextmenu`).

Los eventos `mousedown` y `mouseup` se activan cuando se pulsa y se suelta el botón del ratón, respectivamente, y el evento `mousemove` se activa cada vez que se mueve el ratón. Además, hay varios eventos que se activan dependiendo de si el puntero del ratón se mueve sobre un elemento de la página web o se aleja de él.

Evento	Descripción	Tipo de objeto
`click`	Se activa cuando se pulsa y se suelta el botón del ratón sobre un elemento.	`MouseEvent`
`dblclick`	Se activa cuando se pulsa y se suelta dos veces el botón del ratón sobre un elemento	`MouseEvent`
`contextmenu`	Se activa cuando se abre un menú contextual (normalmente con el botón derecho del ratón).	`MouseEvent`
`auxclick`	Se activa al pulsar un botón del ratón que no es el principal (p. ej., el central).	`MouseEvent`
`mousedown`	Se activa cuando se pulsa el botón del ratón sobre un elemento.	`MouseEvent`

`mouseup`	Se activa cuando se suelta el botón del ratón sobre un elemento	`MouseEvent`
`mousemove`	Se activa cuando se mueve el ratón	`MouseEvent`
`mouseover`	Se activa cuando el ratón se mueve sobre un elemento (véase también el cuadro de notas).	`MouseEvent`
`mouseout`	Se activa cuando el ratón se aleja de un elemento (véase también el cuadro de notas).	`MouseEvent`
`mouseenter`	Se activa cuando el ratón se mueve sobre un elemento (véase también el cuadro de notas).	`MouseEvent`
`mouseleave`	Se activa cuando el ratón se aleja de un elemento (véase también el cuadro de notas).	`MouseEvent`
`wheel`	Se activa al girar la rueda del ratón (sustituye al antiguo mousewheel).	`WheelEvent`

Tabla 3 *Resumen de eventos al interactuar con el ratón*

mouseenter/mouseleave frente a mouseover/mouseout

Los eventos `mouseenter` y `mouseover` se activan cuando el puntero entra en un elemento; `mouseleave` y `mouseout`, cuando sale de él. La diferencia exacta se ilustra en **la figura 3**: `mouseenter` y `mouseleave` solo se activan cuando se cruza el borde exterior del elemento y **no burbujean**, mientras que `mouseover` y `mouseout` se activan además cuando el puntero pasa de un elemento a otro elemento contenido dentro de él (o al revés) y **sí burbujean**.

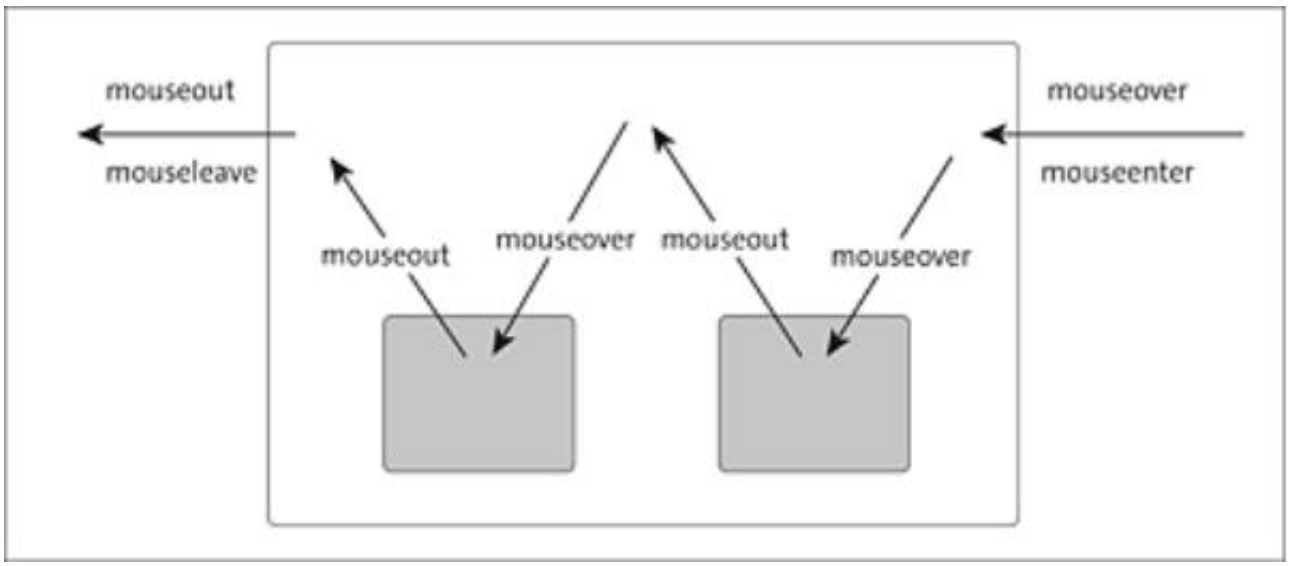


Figura 3 *Comparación de los eventos del ratón mouseenter, mouseout, mouseover y mouseleave*

Además, observará que todos los eventos del ratón son del tipo de objeto `MouseEvent`. Además de las propiedades definidas por el (super)tipo `Event`, este tipo proporciona las propiedades que se muestran en **la tabla 4**.

Propiedad	Descripción
`altKey`	Contiene el valor booleano que indica si se pulsó la tecla (Alt) cuando se activó el evento.
`button`	Contiene el número del botón del ratón que cambió de estado en el evento: 0 (izquierdo), 1 (central) o 2 (derecho).
`buttons`	Contiene un número que representa los botones del ratón pulsados cuando se activó el evento. El número 1 representa el botón izquierdo, el 2 el derecho, el 4 la rueda o el botón central, el 8 el cuarto botón y el 16 el quinto. Si se pulsaron varios botones, buttons contiene la suma de los números respectivos.
`clientX`	La coordenada x (relativa a la ventana visible, el *viewport*) donde se encuentra el puntero del ratón cuando se activa el evento.
`clientY`	La coordenada y (relativa a la ventana visible, el *viewport*) donde se encuentra el puntero del ratón cuando se activa el evento.
`ctrlKey`	Contiene el valor booleano que indica si se pulsó la tecla (Ctrl) cuando se activó el evento.
`metaKey`	Contiene el valor booleano que indica si se pulsó *la tecla meta* cuando se activó el evento (en macOS, la tecla (cmd); en Windows, la tecla (Windows)).
`movementX`	La coordenada x relativa a la coordenada x anterior que se produjo cuando se activó el último evento `mousemove`.
`movementY`	La coordenada y relativa a la coordenada y anterior que se produjo cuando se activó el último evento mousemove.
`offsetX`	La coordenada x relativa al elemento sobre el que se activa el evento (útil para dibujar en un canvas). offsetY es la equivalente para la coordenada y.
`pageX`	La coordenada x relativa al documento completo (incluye el desplazamiento). pageY es la equivalente para la coordenada y.
`relatedTarget`	Contiene el elemento de destino relacionado que está conectado temáticamente con el elemento de destino actual por el evento. Por ejemplo, cuando el puntero del ratón se arrastra desde un elemento a otro, se activa un evento `mouseenter` para este último. La propiedad `relatedTarget` hace referencia al elemento desde el que se arrastró el puntero del ratón.
`screenX`	La coordenada x (relativa a la pantalla) donde se encuentra el puntero del ratón cuando se activa el evento.
`screenY`	La coordenada y (relativa a la pantalla) donde se encuentra el puntero del ratón cuando se activa el evento.

`shiftKey`	Contiene el valor booleano que indica si se pulsó la tecla (Mayús) cuando se activó el evento.

***Tabla 4** Descripción general de las propiedades del tipo de objeto MouseEvent*

Nota

Por razones de espacio, no entraremos en detalle en todas las propiedades de los siguientes tipos de objetos de eventos, ni incluiremos las tablas de descripción general correspondientes. Si busca específicamente los detalles de un tipo de objeto concreto o las distintas API en general, le recomendamos que consulte la página web mencionada anteriormente: evento - API web | MDN

En los dos listados siguientes se muestra un ejemplo de respuesta a eventos del ratón, que se adelanta un poco en cuanto al contenido, pero que resulta muy ilustrativo y ofrece posibilidades para experimentar. Ilustra cómo se puede utilizar una parte de la ventana del navegador como área de dibujo utilizando la API Canvas y el evento `mousemove`.

El listado 18 muestra el código HTML necesario, que sirve de base. El elemento resaltado `<canvas>` representa el área de dibujo. **El listado 19** muestra el código JavaScript que garantiza que, cuando el ratón se mueve sobre el área de dibujo, se dibujan las líneas correspondientes a los movimientos del ratón.

La función `manejarMovimientoRaton()` se registra en el área de dibujo como escuchador del evento `mousemove`. Tan pronto como se mueve el puntero del ratón sobre esta área, este evento se activa en segundo plano y se ejecuta la función. Dentro de ella se accede a las propiedades `offsetX` y `offsetY` del objeto de evento, específicas del tipo `MouseEvent`, que contienen las coordenadas del puntero **relativas al propio elemento `<canvas>`**. (Si se utilizaran `clientX` y `clientY`, las coordenadas serían relativas a la ventana visible y el trazo aparecería desplazado cuando el lienzo no estuviera en la esquina superior izquierda.) Basándose en esa posición, el resto de la función dibuja el trazo en el área de dibujo.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Dibujar según la posición del ratón</title>
  <link rel="stylesheet" href="estilos/principal.css">
  <script src="scripts/principal.js" defer></script>
</head>
<body>
  <div id="contenedor">
    <canvas id="lienzo" width="400" height="400"></canvas>
  </div>
</body>
</html>
```

***Listado 18** Código HTML fuente con área de dibujo*

```
function iniciar() {
  const lienzo = document.getElementById('lienzo');
  const contexto = lienzo.getContext('2d'); // Obtener el área de dibujo
  let iniciado = false;  // ¿Ha comenzado ya el trazo?
```

```
  function manejarMovimientoRaton(evento) {
    const x = evento.offsetX;  // Posición x dentro del lienzo
    const y = evento.offsetY;  // Posición y dentro del lienzo
    if (!iniciado) {  // Si el trazo no ha comenzado...
      contexto.beginPath();  // ...inicia el trazo y...
      contexto.moveTo(x, y);  // ...se sitúa en la posición.
      iniciado = true;
    } else {  // Si el trazo ya ha comenzado...
      contexto.lineTo(x, y);  // ...traza hasta la posición...
      contexto.stroke();  // ...y dibuja la línea.
    }
  }

  lienzo.addEventListener('mousemove', manejarMovimientoRaton);
}
document.addEventListener('DOMContentLoaded', iniciar);
```

Listado 19 *Dibujar en el lienzo utilizando la API Canvas y el evento mousemove*

Pointer Events: un único modelo para ratón, pantalla táctil y lápiz

Los eventos del ratón y los táctiles (**sección 3.6**) se han unificado en los *Pointer Events*, disponibles en todos los navegadores actuales y recomendados hoy: un solo código funciona con ratón, dedo o lápiz. Los principales son `pointerdown`, `pointermove`, `pointerup`, `pointercancel`, `pointerenter`, `pointerleave`, `pointerover` y `pointerout`. Son del tipo `PointerEvent`, que hereda de `MouseEvent` (mantiene `clientX`, `offsetX`, `button`...) y añade `pointerId`, `pointerType` (`'mouse'`, `'touch'` o `'pen'`), `pressure` e `isPrimary`. El método `setPointerCapture()` hace que el elemento siga recibiendo los eventos aunque el puntero salga de él mientras se arrastra.

```
function iniciar() {
  const lienzo = document.getElementById('lienzo');
  const contexto = lienzo.getContext('2d');
  let dibujando = false;

  lienzo.addEventListener('pointerdown', (evento) => {
    dibujando = true;
    lienzo.setPointerCapture(evento.pointerId); // Seguir recibiendo eventos al salir del
lienzo
    contexto.beginPath();
    contexto.moveTo(evento.offsetX, evento.offsetY);
  });

  lienzo.addEventListener('pointermove', (evento) => {
    if (!dibujando) return;  // Solo dibuja con el botón o el dedo pulsado
    contexto.lineTo(evento.offsetX, evento.offsetY);
    contexto.stroke();
  });

  lienzo.addEventListener('pointerup', () => {
    dibujando = false;
  });
```

```
}
document.addEventListener('DOMContentLoaded', iniciar);
```

***Listado 20** Dibujar con Pointer Events (ratón, dedo o lápiz)*

Para que el navegador no interprete el gesto táctil como un desplazamiento de la página, se añade al lienzo la regla CSS `touch-action: none;`.

3.2 Eventos al interactuar con el teclado y con campos de texto

Los eventos también se activan cuando se utiliza el teclado. Cada vez que un usuario pulsa una tecla, se activa `keydown` al pulsarla y `keyup` al soltarla. Antiguamente existía también `keypress`, que representaba el carácter escrito, pero está **obsoleto** y no se dispara con las teclas que no producen caracteres (flechas, Mayús, etc.): para detectar teclas utilice `keydown` y `keyup` y, para conocer el texto introducido, los eventos `beforeinput` e `input` (**sección 3.3**). Los eventos de teclado son del tipo `KeyboardEvent` (véase **la tabla 5**), que proporciona, entre otras, las propiedades `key` (valor de la tecla, p. ej. `'a'`, `'Enter'` o `'ArrowUp'`), `code` (tecla física, p. ej. `'KeyA'`), `repeat` (la tecla se mantiene pulsada) y `altKey`, `ctrlKey`, `metaKey` y `shiftKey`.

Evento	Descripción	Tipo de objeto
`keydown`	Se activa cuando se pulsa una tecla. Si se mantiene pulsada una tecla durante más tiempo, el evento se activa varias veces.	`KeyboardEvent`
`keyup`	Se activa cuando se suelta una tecla.	`KeyboardEvent`
`keypress`	Obsoleto. Se activaba al insertarse un carácter y no se dispara con teclas como las flechas. Utilice keydown o beforeinput/input.	`KeyboardEvent`

***Tabla 5** Resumen de eventos al interactuar con el teclado*

Para ver un ejemplo del uso de eventos de teclado, consulte **el Listado 22**, que muestra cómo mover un elemento de una página web (en el ejemplo, un elemento circular `<div>`, cuyo HTML y CSS mínimos aparecen en el listado anterior) utilizando las teclas de flecha.

Para ello, el evento `keydown` se captura mediante un escuchador de eventos. Dentro del escuchador se determina la tecla pulsada (`evento.key`) y, en función de ella, se llama a una de las funciones `moverArriba()`, `moverAbajo()`, `moverDerecha()` o `moverIzquierda()`. Cada una de ellas actualiza la posición almacenada en el arreglo `posicion` y llama a `mover()`, que a su vez actualiza la posición del elemento mediante CSS.

Observe que, para las teclas de flecha, se llama a `evento.preventDefault()`: de lo contrario el navegador, además de mover el círculo, desplazaría la página. Hasta hace unos años era necesario comprobar también las propiedades `keyIdentifier` y `keyCode` por falta de compatibilidad; hoy `event.key` funciona en todos los navegadores actuales y esas dos propiedades están obsoletas.

```
<style>
  #circulo {
    position: absolute;
```

```
    top: 0;
    left: 0;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    background-color: steelblue;
  }
</style>
<div id="circulo"></div>
```

Listado 21 *Código HTML y CSS del círculo que se moverá con las teclas*

```
function iniciar() {
  const circulo = document.getElementById('circulo');
  const posicion = [0, 0]; // [arriba, izquierda]

  const mover = () => {
    circulo.style.top = `${posicion[0]}px`;
    circulo.style.left = `${posicion[1]}px`;
  };
  const moverArriba = () => { posicion[0] -= 10; mover(); };
  const moverAbajo = () => { posicion[0] += 10; mover(); };
  const moverIzquierda = () => { posicion[1] -= 10; mover(); };
  const moverDerecha = () => { posicion[1] += 10; mover(); };

  window.addEventListener('keydown', (evento) => {
    switch (evento.key) {
      case 'ArrowUp':
        moverArriba();
        break;
      case 'ArrowDown':
        moverAbajo();
        break;
      case 'ArrowLeft':
        moverIzquierda();
        break;
      case 'ArrowRight':
        moverDerecha();
        break;
      default:
        return; // Otras teclas: no hacer nada
    }
    evento.preventDefault(); // Evita que la página se desplace con las flechas
  });
}

document.addEventListener('DOMContentLoaded', iniciar);
```

Listado 22 *Mover un elemento utilizando las teclas de flecha y el evento* `keydown`

Continúa en la UT_04.2

En la siguiente unidad se completa el catálogo de eventos (formularios, foco, interfaz de usuario y dispositivos móviles) y se estudian el flujo de eventos, su interrupción y la activación de eventos mediante programación.